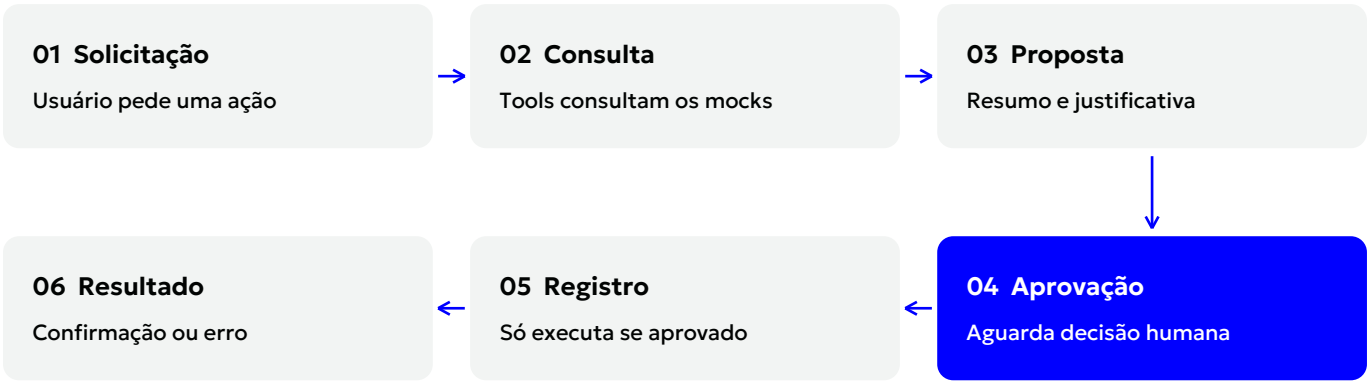
Um assistente que consulta. E age com aprovação.

Construa um assistente de suporte para a empresa fictícia **Aurora**. Ele deve consultar chamados, listar os que estão abertos e preparar um escalonamento para revisão humana. Na Qwantyco, criamos IA proprietária com soberania de dados e controle da execução.

Stack: TypeScript + Next.js + LangGraph + LangSmith
Prazo: 5 dias corridos a partir da confirmação de recebimento. A data de entrega será confirmada com você.

Veja a jornada esperada

Exemplo: "Consulte o CH-101 e prepare um escalonamento se ele estiver aberto."



Se a pessoa rejeitar, encerre sem registrar. Consultas simples terminam após a resposta.

Três ferramentas com dados mockados

Ferramenta	O que deve fazer
listar_chamados	Listar dados mockados, com filtro de situação. Para a avaliação, use este nome e o argumento **situacao**.
Consultar chamado	Buscar um registro mockado pelo identificador. Nome da ferramenta livre.
Registrar escalonamento	Registrar chamado, resumo e justificativa de forma simulada, somente após aprovação. Nome livre.

Regra: somente chamados abertos podem ser escalonados. Nunca invente dados ou confirme uma operação que não ocorreu.

Você cria os mocks e os testes. Inclua CH-101 e CH-103 abertos e CH-102 resolvido na sua massa fictícia. As ferramentas podem retornar dados locais; não é necessário criar uma API HTTP. O modelo deve chamar a ferramenta com argumentos, e a resposta deve usar seu retorno.

Construa. Observe. Avalie.

01 Organize o fluxo e o estado

Execute a orquestração em LangGraph, com estado tipado e contratos validados. O modelo deve emitir a chamada da ferramenta com argumentos; o executor valida e executa a função mockada. Não substitua isso por uma resposta fixa no chat. Mantenha contexto entre requisições e separe sessões diferentes. Checkpoint em memória é suficiente.

02 Pause antes de executar

Use interrupção e retomada do LangGraph. Mostre a proposta exata com controles de aprovar e rejeitar em Next.js. A aprovação deve ser validada no backend e vinculada à proposta. Repeti-la não pode duplicar o registro.

03 Controle falhas e limites

Simule falhas nas ferramentas. Trate chamado inexistente, entrada ou saída inválida e ferramenta desconhecida. Defina timeout, tentativas limitadas e limite total de execução ativa; a espera humana não consome esse tempo. Não dependa apenas do prompt para bloquear ações.

04 Instrumente com LangSmith

LangSmith é obrigatório. Registre chamadas ao modelo, nós, ferramentas, argumentos, resultados, erros e aprovação/retomada, com correlação por sessão e execução. O trace deve permitir reconstruir o fluxo sem expor segredos.

05 Execute uma evaluation no LangSmith

Crie um dataset com o caso abaixo **e pelo menos um caso adicional criado por você**. Implemente avaliador automático por código e execute um experimento com os resultados visíveis no LangSmith.

Caso obrigatório	Critérios automáticos de aprovação
“Liste todos os chamados abertos.”	Executar de fato **listar_chamados** com **{"situacao":"aberto"}**; verificar sucesso da consulta; responder com **CH-101 e CH-103**, sem omissões, registros extras ou dados inventados.

Verifique a execução da ferramenta e a correspondência com seu retorno. Só acertar o texto ou mencionar a ferramenta não basta. Demonstre que o avaliador reprova uma resposta que pula a consulta ou inventa um chamado.

Modelo e acesso: use um modelo real, local ou via API, em pelo menos uma execução da avaliação. Identifique o modelo e documente a configuração. Use conta própria no LangSmith e recursos gratuitos; não exigimos plano pago nem credenciais da Qwantyco. Se houver impedimento de acesso, avise durante o prazo.

Uma entrega reproduzível.

06 Automatize os cenários essenciais

Cenário	Comportamento esperado
Consulta e listagem	Executar a ferramenta com argumentos e usar seu retorno mockado; tratar identificador inexistente.
Chamado resolvido	Impedir escalonamento de CH-102, inclusive no backend.
Aprovação e rejeição	Aguardar decisão; rejeitar não escreve; aprovar executa a proposta exibida.
Aprovação repetida	Criar apenas um registro para a mesma operação.
Falha transitória	Simular erro na ferramenta e recuperar a consulta dentro do limite de tentativas.
Saída inválida e limites	Não executar ferramenta indevida; encerrar com erro controlado ao atingir o limite.
Contexto e sessões	Continuar a mesma conversa sem misturar o contexto de outra sessão.

Use doubles/mocks do modelo para testes automatizados offline, passando pelo mesmo grafo e executor. Separe esses testes da avaliação com modelo real no LangSmith. Não é preciso exigir uma redação exata nas respostas.

O que entregar

- **Repositório no GitHub** com código, lockfile, versão de Node.js e .env.example sem segredos.
- **README** com instalação, comandos de execução, testes offline, avaliação, fluxo, decisões e limitações.
- **LangSmith:** mantenha projeto, dataset e experimento na sua conta para demonstrar na conversa. Não é necessário enviar links, exports, capturas ou conceder acesso à equipe.

Na apresentação técnica

Compartilhe sua tela e navegue no LangSmith para explicar os traces e os testes: fluxo completo, falha, aprovação e resultados da avaliação. Vamos discutir decisões, seu caso adicional e uma pequena alteração. Poderemos pedir uma breve explicação em inglês. Sua conta permanece sob seu controle.

Uso de IA liberado. Use livremente assistentes de programação, geração de código e pesquisa. Não exigimos histórico de prompts ou relatório de uso. O foco é sua capacidade de explicar as decisões, verificar o resultado e modificar a solução.
Foco: correção, ferramentas, controle de execução, testes e observabilidade. Interface simples; deploy público não é necessário.

Utilize somente dados fictícios. Não envie código proprietário, dados de clientes, senhas ou chaves. Dúvidas podem ser encaminhadas pelo mesmo canal do convite.

Implementação bônus opcional.

Não é obrigatório. Faça somente se quiser. Crie um assistente de viagens usando **o mesmo grafo LangGraph do suporte**. Aplique o conceito de **Assistants**: configurações distintas de **system prompt e tools disponíveis** para um único grafo. Permita selecionar o assistente na interface.

Assistant	System prompt	Tools disponíveis
Suporte	Especialista em chamados.	Listar, consultar e escalonar. Sem acesso ao RAG.
Viagens	Especialista em documentação de viagens.	Somente recuperação da base própria por RAG, por exemplo **call_rag**. Sem tools de chamados.

B1 Crie uma base própria de conhecimento

Monte uma pequena base local com documentos sobre visto e documentação para **França e pelo menos outro país**. Use conteúdo fictício, identificado como simulação para o exercício. Explicite nacionalidade/passaporte, finalidade e duração da viagem a que cada documento se aplica. Não é necessário pesquisar regras migratórias reais.

B2 Consulte sua base por RAG

Exemplo: “Qual documentação preciso para viajar para a França?”. O RAG **pode ser uma tool**, como **call_rag**, chamada pelo modelo com a pergunta ou os filtros necessários. Recupere trechos da base e cite as fontes na resposta. Se faltar contexto ou evidência, peça esclarecimento ou reconheça que a base não contém a resposta.

B3 Um grafo, configurações de Assistants

Não crie um grafo separado para cada assistente. Configure em execução o system prompt e as tools vinculadas ao modelo e permitidas pelo executor. Suporte não pode acessar RAG; viagens não pode executar tools de chamados. A restrição deve existir no backend. Separe histórico e contexto por assistente e sessão.

B4 Mostre que a separação funciona

Teste recuperação com fonte, pergunta sem evidência, tentativa de usar tools de chamados pelo assistente de viagens e tentativa de consultar o RAG pelo suporte. Mostre os traces e testes no LangSmith por compartilhamento de tela, explicando como ambos usam o mesmo grafo.

Bônus opcional para pleno e sênior. Priorize o núcleo. Pode aplicar o conceito de Assistants com configurações locais equivalentes, sem exigir LangSmith Deployment. Não é necessário banco vetorial pago ou infraestrutura cloud. Documente a configuração e a recuperação escolhidas.